

Workshop

Basic Python Programming for Statistics and Data Visualization

Jane Marie Pareja

April 14, 2026

1 Statistics

I intended to attend the smart industry workshop today at 10:30 in the Breakout room but I got too absorbed about my chosen project and started working on it that I forgot about the workshop. I missed it. What I did then was look at the workshop materials under smart industry and did not expect to see the basic python programming for statistics[1] and data visualization[2] there. Exactly what I needed for my project.

I opened the presentation, read through it, took my time learning its content and also did the exercises. I downloaded my bank statement and practiced statistics using Python.

In Figure 1.a, I used my old template from my previous project to import the csv to a SQL database via sqlite3 in Python. Figure 1.b shows the raw.db with table named "statements". And in Figure 1.c, I tried getting the mean, median, mode, and standard deviation values of my bank statements which I was able to successfully do so. However, realizing that SQL was very sensitive with its column names, I cleaned the database and used standard column names as illustrated in Figure 1.d.

2 Data Visualization

Next, I read through the next presentation material which was the data visualisation with Python. I learned the content and practiced at the same time using the same data I have which was my statement.

I was trying to figure out where my expenses went the most so Figure 2 showed my first attempt using the plot graph. But it did not make any sense so I used

```

4
5 data_dir = "Data" # folder where the CSV files are located
6 data_file = data_dir + "/raw.db"
7
8 if os.path.exists(data_file): # if the database already exists, delete it
9     print("Removing existing database file", data_file)
10     os.remove(data_file)
11
12 conn = sqlite3.connect(data_file) # open the database
13
14 # check for all files in the folder
15 for file in os.listdir(data_dir):
16     print("Found file with name", file)
17     if file.endswith(".csv"): # if the file is CSV
18         # get the table name from the file name and replace double _ with single _
19         table = file.replace(".csv", "").replace("_", "")
20         # put the file in a dataframe
21         df = pd.read_csv(data_dir + "/" + file)
22         print(df.columns)
23         df["Amount (EUR)"] = df["Amount (EUR)"].str.replace(",", ".", regex=False)
24         df["Amount (EUR)"] = pd.to_numeric(df["Amount (EUR)"], errors="coerce")
25         # add dataframe to the right database table
26         df.to_sql(table, conn, index=False, if_exists="replace")
27     print("Finished loading data into database", data_file)
28 conn.close()

```

Statements	Date	Name / Description	Account	Counterparty	Code	Debit/Credit	Amount (EUR)
20250101	2025-01-01	Bank of America	1234567890123456	ABC	DE	DEBIT	1.00
20250102	2025-01-02	Bank of America	1234567890123456	ABC	DE	DEBIT	1.00
20250103	2025-01-03	Bank of America	1234567890123456	ABC	DE	DEBIT	1.00
20250104	2025-01-04	Bank of America	1234567890123456	ABC	DE	DEBIT	1.00
20250105	2025-01-05	Bank of America	1234567890123456	ABC	DE	DEBIT	1.00

(a) Practice 1

(b) Practice 2

```

1 import sqlite3
2 import statistics
3 import pandas as pd
4 import os
5
6 data_dir = "Data" # folder where the CSV files are located
7 data_file = data_dir + "/raw.db"
8
9 conn = sqlite3.connect(data_file)
10 cursor = conn.cursor()
11
12 data = pd.read_sql_query("SELECT * FROM Statements", conn)
13
14 print("Mean: ", statistics.mean(data["Amount (EUR)"]))
15 print("Median: ", statistics.median(data["Amount (EUR)"]))
16 print("Mode: ", statistics.mode(data["Amount (EUR)"]))
17 print("Standard Deviation: ", statistics.stdev(data["Amount (EUR)"]))
18
19 conn.close()
20
21

```

```

19 conn = sqlite3.connect(data_file)
20 cursor = conn.cursor()
21
22 cursor.execute("DROP TABLE IF EXISTS Statements_clean;")
23
24 cursor.execute("""
25     CREATE TABLE Statements_clean AS
26     SELECT
27         Date,
28         "Name / Description" AS Name_Description,
29         Account,
30         Counterparty,
31         Code,
32         "Debit/Credit" AS Debit_Credit,
33         CAST("Amount (EUR)" AS REAL) AS Amount_eur,
34         "Transaction type" AS Transaction_Type,
35         Notifications,
36         CAST("Resulting balance" AS REAL) AS Resulting_balance,
37         Tag
38     FROM Statements;
39 """)
40
41 data = pd.read_sql_query("SELECT * FROM Statements_clean", conn)
42
43 print("Mean: ", statistics.mean(data["Amount_eur"]))
44 print("Median: ", statistics.median(data["Amount_eur"]))
45 print("Mode: ", statistics.mode(data["Amount_eur"]))

```

(c) Practice 3

(d) Practice 4

Figure 1: Practice Notes

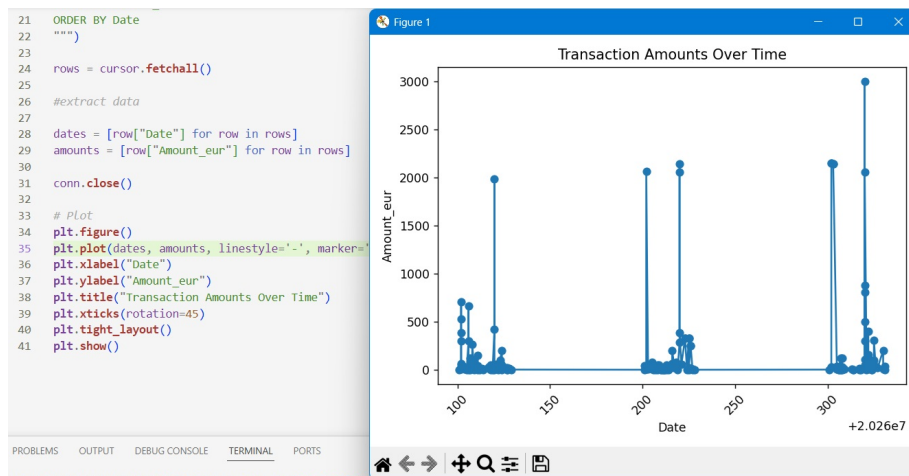


Figure 2: Graph 1

the payment terminal codes on the xlabel to see clearly as shown in Figure 3 using still the plot graph which was everywhere and still did not make any sense. I figured, plot graph is not the best type to use with my goal so I used the bar graph as illustrated in Figure 4. That was more like it.

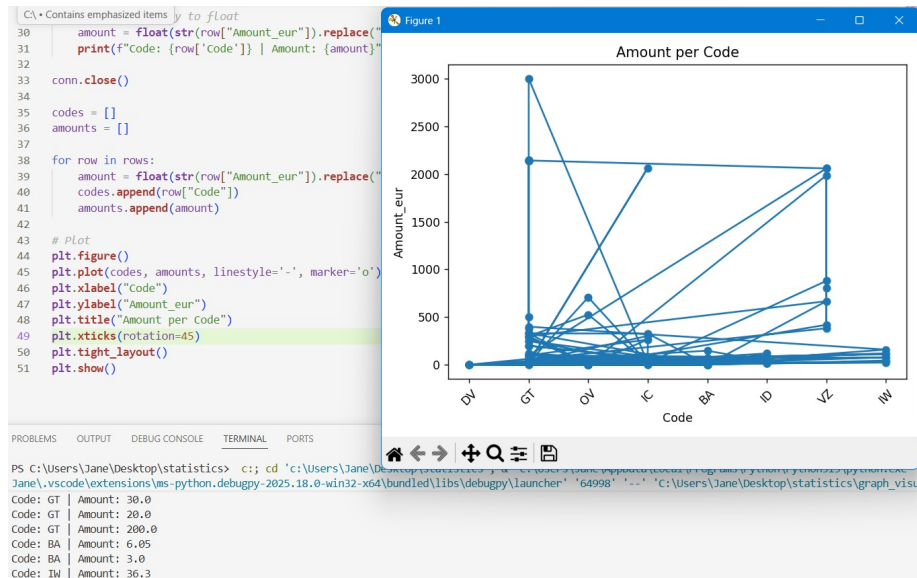


Figure 3: Graph 2

However, I realized that the results were a combination of debit and credit and I only need the debited amounts to get a clear view of my expenses. So in Figure 5, I was trying to count the number of debited transactions only per code and payment terminal using SQL. And when I turned this into a visual, Figure 6 shows a different result.

I also learned about data visualizations at this website Python graph gallery. They provide useful information about graphs and even give suggested programming codes that you can try and experiment with on your own.

3 Reflection

I gained a thorough understanding of how to use statistics and data visualization with Python. It was a very interesting and useful topic that I can use and apply not only with my project but also in my future job. It was really fun to learn new skills and gain new knowledge about the topic that I love because they stay with me. Having to work with data and mostly on the background, it may seem



Figure 4: Graph 3

```

Code,
Transaction_type,
COUNT(*) AS count
FROM Statements_clean
WHERE Debit_credit = 'Debit'
GROUP BY Code
HAVING COUNT(*) > 1;

```

Execute (Ctrl+Enter) History...

Results 7 rows (0.6ms)

Code	Transaction_type	count
BA	Payment terminal	48
DV	Various	6
GT	Online Banking	35
IC	SEPA direct debit	19
ID	iDEAL	6
IW	iDEAL Wero	8
OV	Transfer	28

Figure 5: SQL

like we are doing nothing and only wasting time figuring things out about the database and such, but it was such a fulfilling thing to do.

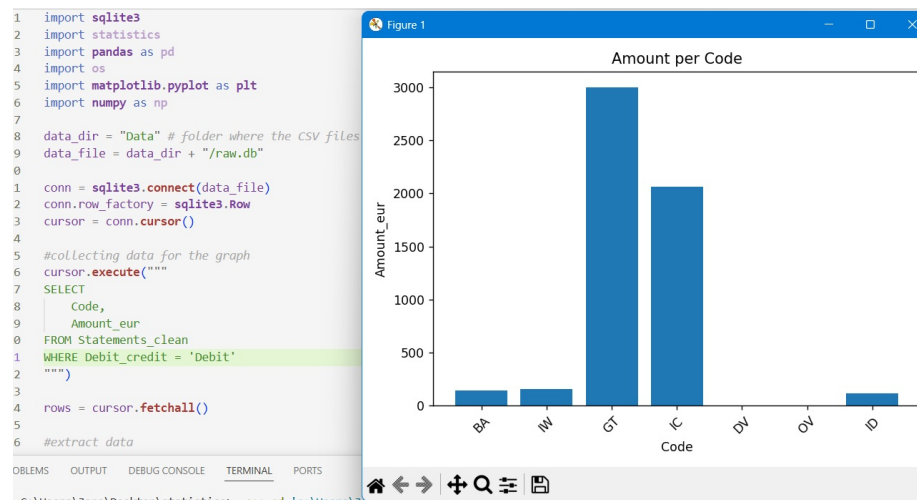


Figure 6: Graph 4

References

- [1] *Basic Statistics with Python*. Fontys University of Applied Sciences. Statistics
- [2] *Data Visualisation with Python*. Fontys University of Applied Sciences. Data visualisation